Roppi

A game of ropes for learning sports-climbing knots

Built with weft, under Literate Driven Development

October 19, 2026

Contents

1 The domain: a game of ropes for climbing knots 4
- 1.1 Purpose . . . 4
- 1.2 The problem . . . 4
- 1.3 Scope . . . 4
- 1.4 Who it is for . . . 5

2 Domain analysis 6
- 2.1 Concepts . . . 6
- 2.2 Invariants . . . 6
- 2.3 Closure of open ends . . . 7
- 2.4 Decided knot choices . . . 7
- 2.5 Vocabulary . . . 7

3 Architecture 9
- 3.1 Decision: rendering in three.js . . . 9
- 3.2 Decision: the rope is a Rapier chain, drawn as one tube . . . 9
- 3.3 Decision: closure is a straight segment between the ends . . . 11
- 3.4 Decision: recognition by planar projection and invariants . . . 11
- 3.5 Decision: anchors are closed components . . . 11
- 3.6 Decision: lessons are data . . . 12
- 3.7 Decision: literate source with weft . . . 12
- 3.8 Decision: tooling . . . 12
- 3.9 Forbidden edges . . . 12
- 3.10 Named seams . . . 13

4 Development stages 14
- 4.1 Stage 1 — the walking skeleton . . . 14
- 4.2 Stage 2 — the recogniser . . . 15
- 4.3 Stage 3 — guided mode for one knot . . . 15
- 4.4 Stage 4 — sandbox . . . 15
- 4.5 Stage 5 — quiz mode . . . 15
- 4.6 Stage 6 — the knot catalogue . . . 15
- 4.7 State of the work . . . 16

5 Scaffold **17**
5.1 Packages . . . 17
5.2 Build and test configuration . . . 18
5.3 Page shell . . . 19

6 Rope model **21**
6.1 Shapes the rest of the code reads . . . 21
6.2 Physics model . . . 22
6.3 Tests . . . 26

7 Topology **31**
7.1 Closure . . . 31
7.2 Crossings and the Fox matrix . . . 31
7.3 Determinant . . . 35
7.4 Recogniser . . . 36
7.5 Tests . . . 37

8 Renderer **40**
8.1 The tube . . . 40
8.2 Tests . . . 42

9 Lessons and the page **44**
9.1 Lesson record . . . 44
9.2 Scene objects . . . 45
9.3 Rope stage . . . 46
9.4 Application . . . 52

10 Recogniser, stage two **54**
10.1 Alexander polynomial . . . 54
10.2 Taught library . . . 57
10.3 Anchors as closed loops . . . 58
10.4 Tests . . . 59

11 Guided mode, stage three **62**
11.1 What a step is . . . 62
11.2 The check for a step . . . 62
11.3 The panel . . . 63
11.4 Tests . . . 64

12 Assembly **66**
12.1 Tooling and page . . . 66
12.2 Rope model . . . 67
12.3 Topology . . . 68
12.4 Recogniser, stage two . . . 68
12.5 Renderer . . . 69

12.6 Guided mode 69

Chapter 1

The domain: a game of ropes for climbing knots

Roppi is a browser game for learning the knots of sports climbing. A player sees a rope drawn in three dimensions, grabs it with the mouse, and moves it. The shapes the rope takes are the knots the player is learning to recognise and tie. The player is a complete beginner.

1.1 Purpose

Roppi teaches climbing knots through hands-on manipulation, not memorised diagrams. A knot is a shape the rope holds, and the shape changes only when the rope is moved. Each knot is connected to its job in the climbing process: why it is tied at that point, and what a mistake looks like.

1.2 The problem

Climbing knots are usually learned from diagrams and from an instructor's hands. A diagram shows one picture of a knot and says nothing about how it is built up or how it can go wrong. A beginner needs to see the rope move through the crossings that make the knot, and to be told when the knot they have built is the one they meant to build. Roppi makes the crossings visible and checks the result.

"Solved" means that a beginner can build each lesson's knot from nothing, and the game names the knot the rope actually forms.

1.3 Scope

In scope:

- A rope rendered in 3D in the browser, moved with the mouse. Touch is not required in the first version.
- A rope model with minimal tension, so the rope feels responsive. It keeps the rope from passing through itself or through an anchor.
- Recognising a named knot by its topology: two shapes are the same knot when one can be dragged into the other without passing through the rope.
- Three teaching modes: guided, sandbox, and quiz.
- The knots of sports climbing as a process. The priority areas are rappel, anchoring, belaying and cleaning. Prusik and other friction hitches are included for their shape only.

Deliberately out of scope:

- Load, holding and slipping behaviour. A knot that would hold or fail under force is not simulated.
- Engineering-accurate rope physics: strength, friction and wear.
- Cutting, splicing and rope joins that change the rope's topology.
- Fishing, sailing and other uses that are not climbing.
- Multiplayer, accounts and online play.
- Certification. Roppi is a proof of concept and a reference. Knots are for learning the shape, and a learner must practise them with a qualified expert. The product says so.

1.4 Who it is for

A complete beginner who wants to learn climbing knots before going on a wall, and who will practise them with an expert.

Chapter 2

Domain analysis

2.1 Concepts

Many climbing knots are tied *around* an anchor object or *join* two ropes. A rope alone cannot express them, so the domain has these concepts.

Rope A flexible curve of finite thickness, with two ends. It is the thing the player moves.

Anchor object A fixed object the rope can be tied around: a bar, a carabiner, a ring, or a tree. All anchor types are in the first version. Without it the topology of a hitch is incomplete.

Second rope A second rope, for bends that join two ropes.

Stopper end The free end of a rope in a rappel setup. A stopper keeps the rope from running through the device.

Knot One or more ropes together with their anchor objects, in a configuration the game recognises. A rope and its anchors count as one knot.

Stage A point in the climbing process: anchoring, belaying, rappel, cleaning, tie-in, or rescue. Each knot belongs to one or more stages.

Lesson A teaching unit: one knot, its stage, its guided steps, and its quiz.

2.2 Invariants

An invariant is a law that, if broken, means the game is no longer about climbing knots.

1. A rope cannot pass through itself or through an anchor object. A knot changes only when the rope is moved through a crossing or around an object.

2. A knot is identified by its topology, with its anchor objects included. Two configurations are the same knot when one can be dragged into the other without passing through the rope or an anchor. The same rope around a bar and around a carabiner can be different knots.

3. A bend joins two rope ends. Its ends must be the ends of the two ropes, not points in the middle. Mixing a bend and a hitch is an error.

4. A stopper knot is valid only at a free rope end. A middle overhand is not a stopper.

5. The rope keeps its length. Moving one part pulls the rest along.

6. A knot stays tied while nothing moves it. Only the player's input and the minimal tension change the shape.

7. Friction hitches are recognised by shape only. The game does not model how they grip under load.

2.3 Closure of open ends

A rope has two free ends, but a topological knot is a closed curve. For recognition, the two ends are joined by a straight segment in space. The segment is never drawn. It is part of the diagram like any other piece of rope. An earlier closure, through a point behind the scene, was dropped. Whether a strand lay in its path depended on how the rope was turned, and it misread a trefoil on about a quarter of random turns. The rope looks open on screen and is closed for recognition.

2.4 Decided knot choices

Rappel stoppers: all options are taught. These are the double overhand, the triple overhand (barrel), and the overhand on a bight.

Joining two ropes for rappel: all options are taught. These are the double overhand and the flat overhand.

Figure-eight on a bight stopper: optional. The lesson notes that a stopper can be added, and does not require it.

2.5 Vocabulary

The book uses these terms, and every chunk name starts with them.

Strand A stretch of rope between two grab points, or between a grab point and an end.

Working end The free end the player is tying with.

Standing part The part of the rope that is not being worked.

Bight A U-shaped loop of rope.

Turn or wrap One full pass of rope around an anchor object or another part of the rope.

Hitch A knot that ties around an object.

Bend A knot that joins two ropes, or two ends of one rope.

Loop A knot that forms a closed loop in the rope.

Friction hitch A hitch that grips a rope under load and slides when unloaded.

Stopper A knot at a rope end that keeps the rope from running through a device.

Closure The way an open rope is closed for topology.

Tension A small pull that keeps the rope settled and responsive.

Chapter 3

Architecture

The system has these parts. Each one becomes a concept chapter later.

`React UI (Tailwind)` — lessons, modes, panels, quiz feedback. It updates on discrete events only, never on each frame.

`Rope model (Rapier)` — rigid links joined by joints.

`Renderer (three.js)` — one continuous tube, anchors and the frame. It is owned by one component.

`Snapshot` — a read-only copy of positions, passed to the topology.

`Topology` — closes the rope, projects it, builds a crossing code, and computes invariants.

`Recogniser` — matches the invariants against the knot library.

`Lessons` — data records that the three modes read.

3.1 Decision: rendering in three.js

We use three.js. The brief asks for it, and it runs in a browser with no install. The alternatives are a game engine such as Unity or Godot, which is heavier and not browser-first, and a 2D canvas, which cannot show the depth that the topology needs.

3.2 Decision: the rope is a Rapier chain, drawn as one tube

The rope is a chain of short rigid links joined neighbour to neighbour, using Rapier. The engine version is pinned at scaffold time.

The length of the rope is kept by a rope joint. This is a maximum-distance constraint, so it allows a little slack. Minimal tension is a weak pull toward a straight line.

The rope cannot pass through itself or through an anchor, because Rapier's contacts keep the links apart. Continuous collision detection is switched on for the links, so that a fast drag does not pass a link through another.

Neighbouring links do collide. This was first planned otherwise: contacts between neighbours were to be disabled, so that a joint alone kept them apart. A joint only limits how far links can separate, so with contacts disabled a lifted link could pass through the one beside it. The rope now keeps contacts between all pairs, and the joint still holds the spacing.

The grab steers one link toward the mouse by setting its velocity each step. It can hold any link. Neither end of the rope is fixed. It is not a joint: a joint pulls with a force the contacts cannot overcome, so a held link passed through a bar. The hand is only a target, so it moves freely, and the held link stops where it meets an obstacle.

Each frame, the link centres are smoothed with a Catmull-Rom curve. The tube's vertices are rewritten into a buffer that is allocated once. The rope therefore never looks like a set of links.

The alternatives considered were these. A custom position-based solver would need segment-to-segment collision written from scratch, which is the hardest part of the rope; Rapier already provides it, and its types confirm the joints and collision controls used here. A Rapier soft-body rope, built with `SoftBodyDesc.rope`, has self-contact options; it is the fallback if the chain proves unstable in stage one. Lines built with `Line2` or `MeshLine` are thin, and forum reports describe memory growth when they are updated each frame. Rebuilding `TubeGeometry` each frame costs an allocation each time.

Gate result. The spike measured the chain before the rest of stage one was built. With no limit, the tail of a forty-link rope dragged at 1.5 m/s reached 8 to 15 m/s, and the soft-body rope did the same. Both were rejected as unstable. The chain is kept with three settings: 32 solver iterations, linear damping of 0.5, and a link speed cap of 3 m/s applied after each physics step. With these, the stretch stays within 1.03 of the rope length, the figure-eight produces no overlap between links, no link enters the post, and a forty-link step costs about 0.7 ms. The cap is a deliberate departure from pure physics: at a hand speed of 1.5 m/s the tail sits at the cap, so the rope moves more slowly than the physics alone would move it. The soft-body rope is not used.

Rendering runs on demand. The frame loop runs while the rope moves and stops when it settles, as the three.js manual advises for scenes that change only on input.

3.3 Decision: closure is a straight segment between the ends

For recognition only, the two free ends are joined by a straight segment, and the loop is read from there. The first choice was a path through a point behind the scene, as the analysis first described. It was dropped: over 200 random turns of a trefoil it misread 49 of them as an unknot, because a strand of the rope could lie in the path, depending on how the rope was turned. The straight segment misread none, and it does not depend on the view.

3.4 Decision: recognition by planar projection and invariants

The closed curve is projected onto the view plane. Each crossing and its over-or-under is read from depth, and the crossings are written as a Planar Diagram code. From that code two invariants are computed: the determinant, counted by Fox colouring, and the Alexander polynomial, from the Wirtinger presentation. A knot is matched against a library only when both invariants agree.

The recogniser reads a snapshot of the link positions. It never changes the rope or the physics world. It runs off the frame loop, throttled, and can move to a Web Worker if profiling shows it blocks frames.

The alternatives were these. Matching the pose against stored templates fails invariant 2, because the pose changes with every drag. The determinant alone cannot separate some knots.

The invariants do not separate every pair of knots, and they cannot tell a mirror image from its original. The library is therefore checked for collisions at build time, and the taught set is checked before lesson content is written.

3.5 Decision: anchors are closed components

Each anchor is a closed loop in space. A carabiner, a bar, a ring and a tree trunk are all loops. Anchors are separate components, so a hitch around an anchor is a link between the rope and the anchor. The recogniser handles several components in the same way as one.

Modelling an anchor as a fixed point cannot represent a rope wrapped around it. Modelling the anchor inside the rope merges two topologies, which is wrong.

3.6 Decision: lessons are data

Each knot is one data record. It holds the name, the stage, the web sources, the invariants, the guided steps and the quiz prompt. Guided, sandbox and quiz modes all read the same records. Quiz evaluation compares the recognised knot with the prompt. A hard-coded lesson would turn every change into a code change, and a remote content service is not needed for a proof of concept.

3.7 Decision: literate source with weft

The source of truth is these `.weft` chapters. The `weft` tool tangles them into `src/`. We built weft 2.0.2 from its repository and copied the binary to the user's local bin directory.

3.8 Decision: tooling

Vite is the dev server and build tool. React is the UI. Tailwind styles the UI only, never the 3D canvas. Vitest runs the tests. three.js and Rapier are pinned to exact versions.

The 3D scene is owned by one component. That component creates the renderer, the scene and the physics world in an effect, and disposes of them on unmount.

The research behind the rendering choices comes from the three.js manual, which is in the three.js source repository. Its advice is these. Merging geometry reduces draw calls. Geometries, materials and textures are not freed automatically, so `dispose()` must be called when they are no longer used. A continuously animated loop is wasteful for a scene that changes only on input.

3.9 Forbidden edges

These are rules that a change must not break. They are checked in review.

1. The rope model does not import three.js.
2. The renderer never writes rope state.
3. The recogniser reads a snapshot and never changes the rope, the physics world or the scene.
4. Topology does not import React or three.js.
5. React holds no per-frame state. Per-frame data is written to refs.

6. Lesson records are data, with no code and no three.js or Rapier objects.
7. Anchors are scene data. The rope model does not own anchor geometry.
8. One component creates and disposes of the physics world and the renderer. No other code does.
9. Tailwind does not style the canvas.

3.10 Named seams

These contracts are empty until stage one.

RopeSnapshot The link centres, the radius and the two ends. Read-only.

AnchorSnapshot A closed loop of points, one per anchor.

KnotResult A name, or nothing if the knot is not recognised, together with its invariants.

LessonRecord The data described above.

Chapter 4

Development stages

Each stage is an edition of the project. At its boundary everything builds, the tests pass, and the stage's "proven when" fact holds. No stage begins before the previous one is proven.

4.1 Stage 1 — the walking skeleton

The thinnest slice that runs. It touches every stage-one concept, however thinly. The concepts are the rope model, one anchor (a bar), the closure, the topology (the determinant only), a two-entry recogniser (unknot and trefoil), the renderer, a React page with Tailwind and one lesson stub, and one lesson record. Tooling is Vite, Vitest and the pinned libraries.

Decision gate. Before the rest of stage one is built, the rigid chain is measured for stability, for no-pass behaviour, and for frame cost. The gate passed on 2026-10-19 with the rigid chain and a link speed cap. The soft-body rope was measured and not used. The measurements and settings are recorded in the rope decision of the architecture chapter.

Proven when the page runs; a rope can be dragged with the mouse and draws continuously; and a test that drives the rope into a trefoil returns `trefoil`, while a test of the unknot returns `unknot`.

Status. Proven on 2026-10-19. The page builds and runs in a headless browser, and the rope can be dragged. A rope tied by hand into an overhand knot is named `trefoil` by the page. The unit tests also feed the recogniser a scripted curve, which covers the same knot without a person tying it.

4.2 Stage 2 — the recogniser

Adds the Alexander polynomial, the full library for the lessons, anchors of every type, and the build-time collision check.

Proven when every library entry is recognised from a scripted rope of that knot; every pair in the library is separated by the two invariants; and a clove hitch on a bar is recognised as a link with the bar.

4.3 Stage 3 — guided mode for one knot

Guided mode for overhand on a bight, with steps. The UI waits for the recogniser to confirm each step.

Proven when a guided lesson runs from its first step to the recognised knot, driven by the recogniser and not by a timer.

Status. Proven on 2026-10-19. A person followed the overhand lesson on the page, and all four steps were detected in the order the lesson gives. The scripted tests cover the same path without a person.

4.4 Stage 4 — sandbox

Free play. The recogniser names any library knot and says nothing otherwise.

Proven when a rope tied into each library knot is named in sandbox mode, and an unknown configuration is left unnamed.

4.5 Stage 5 — quiz mode

The instruction is shown, the player ties the knot, and asks for an evaluation.

Proven when a correct tie is marked correct; a wrong knot is marked wrong and named as the knot the player actually tied; and an untied rope is not evaluated.

4.6 Stage 6 — the knot catalogue

Adds the lessons from the research, in teaching order. Anchoring: figure-eight on a bight, clove hitch, girth hitch. Belaying: munter hitch, mule

overhand. Rappel: the stoppers and the rope joins, as decided in the analysis. Cleaning: the rappel stoppers and backups, with no separate knot. Other: prusik and the friction hitches, shape only.

Proven when each lesson has a scripted reference rope that the recogniser names, and each lesson cites its web sources.

4.7 State of the work

Lifecycle: planned, drafted, implemented, stable.

Concept	Status	Stage	Chapter
Rope model	implemented	1	Rope model
Anchors	implemented	1 (bar), 2 (all)	Lessons and the page
Closure	implemented	1	Topology
Topology	implemented	1 (determinant), 2 (Alexander)	Topology
Recogniser	implemented	1 (two entries), 2 (library)	Topology
Renderer	implemented	1	Renderer
UI shell	implemented	1	Lessons and the page
Lesson record	implemented	1	Lessons and the page
Guided mode	stable	3	Guided mode, stage three
Sandbox mode	planned	4	—
Quiz mode	planned	5	—
Knot catalogue	planned	6	—

Chapter 5

Scaffold

Stage one needs a build tool, a test runner, a UI framework, and the two pinned libraries. Every version is exact. A rope that behaves today must behave the same after a dependency update, so nothing uses a caret or a tilde.

All code lives in the `app` directory, which the assembly routes to. The book stays at the root. The package manifest is written here and tangled out. The lock file is produced by npm when the packages are installed, and it is committed with the manifest.

5.1 Packages

Runtime: three.js renders, Rapier simulates, React draws the page. Development: Vite builds, Vitest tests, Tailwind styles the page (never the canvas), and TypeScript checks types. The versions were the latest releases at scaffold time.

⟨ *package manifest* 17 ⟩ ≡

```
{
  "name": "roppi",
  "version": "0.1.0",
  "private": true,
  "description": "a game of ropes using ldd",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc --noEmit && vite build",
    "test": "vitest run",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@dimforge/rapier3d-compat": "0.21.0",
    "react": "19.3.1",
    "react-dom": "19.3.1",
```

```
    "three": "0.186.1"
  },
  "devDependencies": {
    "@tailwindcss/vite": "4.3.3",
    "@types/react": "19.3.0",
    "@types/react-dom": "19.3.0",
    "@types/three": "0.186.0",
    "@vitejs/plugin-react": "6.1.2",
    "tailwindcss": "4.3.3",
    "typescript": "5.9.3",
    "vite": "8.3.4",
    "vitest": "5.0.5"
  }
}
```

□

17 · used in 66a

5.2 Build and test configuration

Vite serves the page and builds it. The same file configures Vitest. Tests run in plain Node, because the topology and the rope model need no browser. The renderer is tested for its geometry, and the page itself is checked by running it.

⟨ *Vite configuration* 18a ⟩ ≡

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
  test: {
    environment: 'node',
    // The rope runs four physics substeps per frame, so its tests
    take seconds.
    testTimeout: 20000,
    include: ['src/**/*.test.ts'],
  },
});
```

□

18a · used in 66b

⟨ *TypeScript configuration* 18b ⟩ ≡

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
```

```
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "noEmit": true,
    "isolatedModules": true,
    "skipLibCheck": true
  },
  "include": ["src", "vite.config.ts"]
}
```

□

18b · used in 66c

5.3 Page shell

The page has one root element. React mounts into it and nothing else is written into the document.

⟨ *HTML shell* 19a ⟩ ≡

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale
    =1.0" />
    <title>Roppi</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

□

19a · used in 66d

⟨ *Global styles* 19b ⟩ ≡

```
@import "tailwindcss";

html,
body,
#root {
  height: 100%;
  margin: 0;
}
```

□

19b · used in 67a

⟨ *Entry point* 19c ⟩ ≡

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { App } from './App';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

□

19c · used in 67b

Chapter 6

Rope model

The rope is a chain of eighty links, each a small ball, joined neighbour to neighbour. Each joint keeps neighbouring links within one link spacing of each other. Every pair of links collides, neighbours included. A joint alone only stops links from moving apart, so without neighbour contact a link could pass through the one beside it.

The links are drawn as one continuous tube through their centres, so the spacing has to be small against the radius. With forty links at 2.5 radii apart, the straight piece between two neighbours came within half a diameter of a piece further along the rope while every pair of centres stayed clear. The rope visibly passed through itself when a knot was pulled tight. Eighty links at 1.25 radii keep the tube from crossing itself under the same pulls, for the same length of rope and about twice the physics cost (1.5 ms a frame).

Both ends are free. Nothing holds the rope in place, so the player moves it by grabbing any link. Gravity is a switch: off by default, so the rope floats and can be tied in the air, and on, so the rope falls onto the floor. The rope model knows nothing of three.js or React.

The numbers come from the decision gate in the architecture chapter. Solver iterations, damping and the speed cap are the values that passed it. The cap is applied after every physics step. It is a deliberate departure from pure physics, and the gate result records why.

6.1 Shapes the rest of the code reads

The rope model returns a snapshot: the link centres in order, from the first link to the last, and the link radius. Anchors and the floor are boxes in the scene. The model takes them as plain shapes, so it does not depend on what they are for.

⟨ *Rope: snapshot types* 21 ⟩ ≡

```
export interface Vec3 {
  x: number;
  y: number;
  z: number;
}

// The link centres in order, from the first link to the last.
export interface RopeSnapshot {
  readonly points: readonly Vec3[];
  readonly radius: number;
}

// An axis-aligned box given by its centre and its half sizes.
export interface Box {
  center: Vec3;
  half: Vec3;
}
```

□

21 · used in 67g

6.2 Physics model

The physics engine is loaded once and shared. A rope is created in a world of its own, so disposing of a rope frees exactly what it made.

Each link is a dynamic ball with continuous collision detection on, so a fast drag cannot pass a link through another. The physics runs in four substeps per frame. Measured on a trefoil falling onto the floor, one substep let two links that are not neighbours come within 0.029 m of each other, where contact is 0.040 m. That is a pass-through, and it let the knot slip apart. Four substeps keep the closest pair at 0.039 m. Linear damping slows each link, so a released rope settles instead of wobbling. Measured after a release, a damping of 0.5 took about six seconds to settle, 2 took under two, and 4 took under one. The value is 4. The speed cap is applied to every link after the step, and the step returns the fastest link speed, which the page uses to decide when the rope has settled.

A grab steers the held link toward the hand by setting its velocity before each step. It is not a joint. A joint pulls its link with a force the contacts cannot overcome, so a held link would pass through a bar it was pushed against. A velocity does give way to a contact, so the link stops at the bar. The grab starts at the link's current position, so picking a link never moves the rope. Releasing stops the steering. The rest of the rope follows the held link through the joints, as a real rope follows a hand.

The held point moves toward the hand at no more than 1.5 m/s, and the link is steered toward it. Steering the link straight to the hand is what made the rope turbulent. On a 0.3 m flick of the mouse, the peak link speed was 1.3 m/s when steered straight to the hand, and 0.7 m/s with the limit. The

limit is fast enough to follow a quick hand, so the held point stays under the cursor.

⟨ *Rope: physics model* 23 ⟩ ≡

```
import RAPIER from '@dimforge/rapier3d-compat';
import type { Box, RopeSnapshot, Vec3 } from './snapshot';

export const LINK_COUNT = 80;
export const LINK_RADIUS = 0.02;
export const LINK_SPACING = 0.025;
export const MAX_LINK_SPEED = 3;
const SOLVER_ITERATIONS = 32;
// Physics steps per frame. Links move about a diameter per step
    when the rope falls,
// and with one step they pass into each other. Four steps stop that
    .
const SUBSTEPS = 4;
const LINEAR_DAMPING = 4;
// Each link has half the density it had at forty links, so the
    whole rope weighs
// the same as before. Eighty links at full density made the rope
    feel twice as heavy.
const LINK_DENSITY = 0.5;
// The held point follows the hand no faster than this, so a quick
    flick cannot throw the rope.
const HAND_SPEED = 1.5;
// The held link is steered toward the held point at this rate, per
    second.
const STEER_GAIN = 8;
const GRAVITY = -9.81;

export interface Rope {
  grabLink(index: number, target: Vec3): void;
  moveGrab(target: Vec3): void;
  releaseGrab(): void;
  setGravity(on: boolean): void;
  step(): number;
  snapshot(): RopeSnapshot;
  dispose(): void;
}

let ready: Promise<void> | null = null;

export function initPhysics(): Promise<void> {
  ready ??= RAPIER.init();
  return ready;
}

// A rope lying straight from start along the x axis, with its first
     link at start.
// With a pose, the links start at the given centres instead; the
    tests use this to start from a knot.
export function createRope(start: Vec3, obstacles: readonly Box[],
```

```
  pose?: Vec3[]): Rope {
const world = new RAPIER.World({ x: 0, y: 0, z: 0 });
world.numSolverIterations = SOLVER_ITERATIONS;
world.timestep = 1 / 60 / SUBSTEPS;

const links: RAPIER.RigidBody[] = [];
for (let i = 0; i < LINK_COUNT; i++) {
  const at = pose?.[i] ?? { x: start.x + i * LINK_SPACING, y:
  start.y, z: start.z };
  const body = world.createRigidBody(
    RAPIER.RigidBodyDesc.dynamic()
      .setTranslation(at.x, at.y, at.z)
      .setLinearDamping(LINEAR_DAMPING)
      .setCcdEnabled(true),
  );
  world.createCollider(RAPIER.ColliderDesc.ball(LINK_RADIUS).
  setDensity(LINK_DENSITY), body);
  links.push(body);
}

const origin = { x: 0, y: 0, z: 0 };
for (let i = 0; i < LINK_COUNT - 1; i++) {
  world.createImpulseJoint(RAPIER.JointData.rope(LINK_SPACING,
  origin, origin), links[i], links[i + 1], true);
}

for (const box of obstacles) {
  const body = world.createRigidBody(
    RAPIER.RigidBodyDesc.fixed().setTranslation(box.center.x, box.
  center.y, box.center.z),
  );
  world.createCollider(RAPIER.ColliderDesc.cuboid(box.half.x, box.
  half.y, box.half.z), body);
}

// The held link, where the hand has asked it to be (goal), and
  the point it is
// steered toward (point), which follows the goal no faster than
  HAND_SPEED.
let held: { index: number; goal: Vec3; point: Vec3 } | null = null
  ;

const releaseGrab = () => {
  held = null;
};

// Steers the held link toward its goal by setting its velocity. A
   contact can
// still take that velocity away, so the link stops at a bar
  instead of passing it.
const steerHeld = () => {
  if (!held) return;
  const link = links[held.index];
  const at = link.translation();
  const dt = world.timestep;
```

```
  const { goal, point } = held;
  const dx = goal.x - point.x;
  const dy = goal.y - point.y;
  const dz = goal.z - point.z;
  const gap = Math.hypot(dx, dy, dz);
  if (gap > 0) {
    const move = Math.min(gap, HAND_SPEED * dt) / gap;
    point.x += dx * move;
    point.y += dy * move;
    point.z += dz * move;
  }
  const v = {
    x: STEER_GAIN * (point.x - at.x),
    y: STEER_GAIN * (point.y - at.y),
    z: STEER_GAIN * (point.z - at.z),
  };
  const speed = Math.hypot(v.x, v.y, v.z);
  if (speed > MAX_LINK_SPEED) {
    const scale = MAX_LINK_SPEED / speed;
    v.x *= scale;
    v.y *= scale;
    v.z *= scale;
  }
  link.setLinvel(v, true);
};

return {
  grabLink(index, target) {
    held = { index, goal: { ...target }, point: { ...links[index].
  translation() } };
  },
  moveGrab(target) {
    if (held) held.goal = { ...target };
  },
  releaseGrab,
  setGravity(on) {
    world.gravity = on ? { x: 0, y: GRAVITY, z: 0 } : { x: 0, y:
  0, z: 0 };
    // A settled link sleeps, and a sleeping link ignores a change
   of gravity until it wakes.
    for (const body of links) body.wakeUp();
  },
  step() {
    let fastest = 0;
    for (let k = 0; k < SUBSTEPS; k++) {
      steerHeld();
      world.step();
      for (const body of links) {
        const v = body.linvel();
        const speed = Math.hypot(v.x, v.y, v.z);
        if (speed > MAX_LINK_SPEED) {
          const scale = MAX_LINK_SPEED / speed;
          body.setLinvel({ x: v.x * scale, y: v.y * scale, z: v.z
  * scale }, true);
        }
```

```
          fastest = Math.max(fastest, Math.min(speed, MAX_LINK_SPEED
    ));
        }
      }
      return fastest;
    },
    snapshot() {
      return { points: links.map((body) => body.translation()),
    radius: LINK_RADIUS };
    },
    dispose() {
      world.free();
    },
  };
}
```

□

23 · used in 68a

6.3 Tests

The tests carry the gate criteria forward, so a later change cannot quietly break the rope. A link may stretch by at most five per cent of the spacing. A drag must keep every link under the speed cap. A held link must not pass through a bar. A released rope must come to rest, which is the check against wobble.

⟨ *Rope: tests* 26 ⟩ ≡

```
import { beforeAll, describe, expect, it } from 'vitest';
import { LINK_COUNT, LINK_RADIUS, LINK_SPACING, MAX_LINK_SPEED,
    createRope, initPhysics } from './rope';
import type { Vec3 } from './snapshot';

const START: Vec3 = { x: -0.975, y: 0.2, z: 0 };
const FLOOR = { center: { x: 0, y: -1, z: 0 }, half: { x: 2, y:
    0.05, z: 2 } };

// The standard trefoil curve as a pose for the rope, its links
    spaced evenly along it.
function trefoilPose(): Vec3[] {
  const f = (t: number): Vec3 => ({
    x: Math.sin(t) + 2 * Math.sin(2 * t),
    y: Math.cos(t) - 2 * Math.cos(2 * t),
    z: -Math.sin(3 * t),
  });
  const N = 20000;
  const T = 2 * Math.PI * 0.97;
  const pts = Array.from({ length: N + 1 }, (_, i) => f((T * i) / N)
    );
  const cum = [0];
  for (let i = 1; i <= N; i++) {
```

```
    const a = pts[i - 1];
    const b = pts[i];
    cum.push(cum[i - 1] + Math.hypot(b.x - a.x, b.y - a.y, b.z - a.z
    ));
  }
  const scale = (LINK_SPACING * (LINK_COUNT - 1)) / cum[N];
  const out: Vec3[] = [];
  for (let k = 0; k < LINK_COUNT; k++) {
    const target = k * LINK_SPACING;
    let j = 1;
    while (j < N && cum[j] * scale < target) j++;
    const p = pts[j];
    out.push({ x: p.x * scale, y: p.y * scale, z: p.z * scale });
  }
  return out;
}

// The largest neighbour distance, divided by the spacing. One means
     taut.
function longestStretch(points: readonly Vec3[]): number {
  let longest = 0;
  for (let i = 0; i < points.length - 1; i++) {
    const a = points[i];
    const b = points[i + 1];
    longest = Math.max(longest, Math.hypot(a.x - b.x, a.y - b.y, a.z
     - b.z) / LINK_SPACING);
  }
  return longest;
}

beforeAll(async () => {
  await initPhysics();
});

describe('rope model', () => {
  it('lies still with gravity off and no link stretched beyond five
    per cent', () => {
    const rope = createRope(START, []);
    for (let s = 0; s < 300; s++) rope.step();
    const { points } = rope.snapshot();
    expect(points).toHaveLength(LINK_COUNT);
    expect(longestStretch(points)).toBeLessThan(1.05);
    expect(rope.step()).toBeLessThan(0.01);
    rope.dispose();
  });

  it('falls onto the floor with gravity on', () => {
    const rope = createRope(START, [FLOOR]);
    rope.setGravity(true);
    for (let s = 0; s < 600; s++) rope.step();
    const lowest = Math.min(...rope.snapshot().points.map((p) => p.y
    ));
    expect(lowest).toBeGreaterThan(FLOOR.center.y + FLOOR.half.y -
    0.01);
```

```
  expect(lowest).toBeLessThan(FLOOR.center.y + FLOOR.half.y +
  LINK_RADIUS + 0.01);
  expect(longestStretch(rope.snapshot().points)).toBeLessThan
  (1.05);
  rope.dispose();
});

it('falls when gravity is switched on after it has settled', () =>
   {
  const rope = createRope(START, [FLOOR]);
  for (let s = 0; s < 600; s++) rope.step();
  const settled = Math.min(...rope.snapshot().points.map((p) => p.
  y));
  rope.setGravity(true);
  for (let s = 0; s < 300; s++) rope.step();
  expect(Math.min(...rope.snapshot().points.map((p) => p.y))).
  toBeLessThan(settled - 0.1);
  rope.dispose();
});

it('does not let links pass through each other when a knot falls',
   () => {
  const rope = createRope({ x: 0, y: 0, z: 0 }, [FLOOR],
  trefoilPose());
  for (let s = 0; s < 300; s++) rope.step();
  rope.setGravity(true);
  let closest = Infinity;
  for (let s = 0; s < 600; s++) {
    rope.step();
    const p = rope.snapshot().points;
    for (let i = 0; i < p.length; i++) {
      for (let j = i + 2; j < p.length; j++) {
        closest = Math.min(closest, Math.hypot(p[i].x - p[j].x, p[
  i].y - p[j].y, p[i].z - p[j].z));
      }
    }
  }
  // Contact is two radii apart. Allowing five per cent of overlap
   is generous.
  expect(closest).toBeGreaterThan(2 * LINK_RADIUS * 0.95);
  rope.dispose();
});

it('picks up a link without moving it', () => {
  const rope = createRope(START, []);
  for (let s = 0; s < 300; s++) rope.step();
  const before = rope.snapshot().points[20];
  rope.grabLink(20, before);
  rope.step();
  const after = rope.snapshot().points[20];
  expect(Math.hypot(after.x - before.x, after.y - before.y, after.
  z - before.z)).toBeLessThan(0.005);
  rope.releaseGrab();
  rope.dispose();
});
```

```
it('keeps every link under the speed cap while a middle link is
  dragged in a circle', () => {
  const rope = createRope(START, []);
  for (let s = 0; s < 300; s++) rope.step();
  const start = rope.snapshot().points[20];
  rope.grabLink(20, start);
  const radius = 0.4;
  const omega = 1.5 / radius;
  let fastest = 0;
  for (let s = 0; s < 360; s++) {
    const t = s / 60;
    rope.moveGrab({
      x: start.x + radius * Math.sin(omega * t),
      y: start.y,
      z: start.z + radius * (1 - Math.cos(omega * t)),
    });
    fastest = Math.max(fastest, rope.step());
  }
  expect(fastest).toBeLessThanOrEqual(MAX_LINK_SPEED);
  expect(longestStretch(rope.snapshot().points)).toBeLessThan
  (1.05);
  rope.dispose();
});

it('comes to rest after it is released', () => {
  const rope = createRope(START, []);
  for (let s = 0; s < 300; s++) rope.step();
  const start = rope.snapshot().points[20];
  rope.grabLink(20, start);
  for (let s = 0; s < 120; s++) {
    rope.moveGrab({ x: start.x, y: start.y + 0.3 * Math.sin(s /
  20), z: start.z });
    rope.step();
  }
  rope.releaseGrab();
  let fastest = 1;
  for (let s = 0; s < 600; s++) fastest = rope.step();
  expect(fastest).toBeLessThan(0.02);
  rope.dispose();
});

it('does not let a quick flick of the mouse throw the rope', () =>
   {
  const rope = createRope(START, []);
  for (let s = 0; s < 300; s++) rope.step();
  const p0 = rope.snapshot().points[20];
  rope.grabLink(20, p0);
  let peak = 0;
  for (let s = 0; s < 240; s++) {
    // The hand jumps 0.3 m in one frame, then stays still.
    rope.moveGrab({ x: p0.x + (s >= 10 ? 0.3 : 0), y: p0.y, z: 0
  });
    peak = Math.max(peak, rope.step());
  }
```

```
    expect(peak).toBeLessThan(1);
    rope.dispose();
  });

  it('does not pass a held link through a bar', () => {
    // A bar lies across the rope's path at its height, and the held
     link is pushed up through it.
    const held = 25;
    const x = START.x + held * LINK_SPACING;
    const bar = { center: { x, y: 0.4, z: 0 }, half: { x: 0.02, y:
    0.02, z: 0.4 } };
    const rope = createRope(START, [bar]);
    for (let s = 0; s < 300; s++) rope.step();
    const start = rope.snapshot().points[held];
    rope.grabLink(held, start);
    let highest = -Infinity;
    for (let s = 0; s < 180; s++) {
      rope.moveGrab({ x: start.x, y: 0.8, z: 0 });
      rope.step();
      highest = Math.max(highest, rope.snapshot().points[held].y);
    }
    // The bar's underside is at y = 0.38, so a link under it stays
    below 0.38 - radius of contact.
    expect(highest).toBeLessThan(bar.center.y - bar.half.y);
    rope.dispose();
  });
});
```

□

26 · used in 68b

Chapter 7

Topology

Recognition works on the shape of the rope, not on its pose. Three steps make that possible. The rope is closed, projected, and read as a knot diagram. Then one invariant is computed from the diagram. The determinant is the simplest, and it is the only one stage one uses.

7.1 Closure

A rope has two free ends, and a knot needs a closed loop. The ends are joined by a straight segment, from the last link back to the first, and the loop is read from there. The segment is part of the diagram like any other piece of rope.

The first closure joined the ends through a point far behind the background. It was dropped. Its path runs straight back from each end, so whether a strand of the rope lay in that path depended on how the rope was turned. Over 200 random turns of a trefoil, it misread 49 of them as an unknot. The straight segment misread none. Over 400 turns of a trefoil with open ends, the straight segment misread none. It also does not depend on the view.

7.2 Crossings and the Fox matrix

The viewer looks down the depth axis, so the diagram is the rope's projection onto the x and y plane. Two segments that cross in that plane form a crossing. The one with the greater depth is over. Depth is compared at the crossing point itself, not at the segment ends.

Every crossing has one under-passage, which splits the under-strand into arcs. Walking along the loop, arc j runs from under-passage j to under-passage $j + 1$. Each crossing touches three arcs: the arc entering its under-passage, the arc leaving it, and the arc that holds the over-strand. Fox colouring says that for every crossing, twice the over colour minus the two

under colours is zero modulo the determinant. The coefficients of that condition form one row of the Fox matrix, and the determinant is read from that matrix.

⟨ *Topology: crossings and Fox matrix* 32 ⟩ ≡

```
import type { Vec3 } from '../rope/snapshot';

const EPS = 1e-12;

// One crossing of the projected diagram. The over strand passes
    above the under strand.
// A passage is a component, a segment of that component, and a
    fraction along the segment.
export interface Crossing {
  overComponent: number;
  overSegment: number;
  overAt: number;
  underComponent: number;
  underSegment: number;
  underAt: number;
  // The handedness of the crossing, from the directions of its two
    strands in the view.
  sign: 1 | -1;
}

// A diagram of closed components, seen down the z axis.
export interface Diagram {
  components: number;
  crossings: Crossing[];
  // How many under-passages each component has. Each one cuts the
    component into arcs.
  underCount: number[];
}

// Where segment a-b meets segment c-d in the view, as fractions
    along each, or null.
// Parallel or degenerate segments never meet.
function meet(a: Vec3, b: Vec3, c: Vec3, d: Vec3): { s: number; u:
    number } | null {
  const rx = b.x - a.x;
  const ry = b.y - a.y;
  const wx = d.x - c.x;
  const wy = d.y - c.y;
  const denom = rx * wy - ry * wx;
  if (Math.abs(denom) < EPS) return null;
  const qx = c.x - a.x;
  const qy = c.y - a.y;
  const s = (qx * wy - qy * wx) / denom;
  const u = (qx * ry - qy * rx) / denom;
  if (s < 0 || s >= 1 || u < 0 || u >= 1) return null;
  return { s, u };
}
```

```
// The crossings of closed components. A component is a list of
    points, and its last
// point joins back to its first. Segments of one component that
    touch are skipped.
export function diagramOf(components: readonly (readonly Vec3[])[]):
     Diagram {
  const crossings: Crossing[] = [];
  for (let ci = 0; ci < components.length; ci++) {
    for (let cj = ci; cj < components.length; cj++) {
      const a = components[ci];
      const b = components[cj];
      for (let i = 0; i < a.length; i++) {
        const start = ci === cj ? i + 2 : 0;
        for (let j = start; j < b.length; j++) {
          if (ci === cj && i === 0 && j === b.length - 1) continue;
    // these two share a vertex
          const p = a[i];
          const q = a[(i + 1) % a.length];
          const r = b[j];
          const s = b[(j + 1) % b.length];
          const hit = meet(p, q, r, s);
          if (!hit) continue;
          const depthI = p.z + hit.s * (q.z - p.z);
          const depthJ = r.z + hit.u * (s.z - r.z);
          const first = { component: ci, segment: i, at: hit.s, dx:
    q.x - p.x, dy: q.y - p.y };
          const second = { component: cj, segment: j, at: hit.u, dx:
     s.x - r.x, dy: s.y - r.y };
          const [over, under] = depthI > depthJ ? [first, second] :
    [second, first];
          crossings.push({
            overComponent: over.component,
            overSegment: over.segment,
            overAt: over.at,
            underComponent: under.component,
            underSegment: under.segment,
            underAt: under.at,
            sign: over.dx * under.dy - over.dy * under.dx > 0 ? 1 :
    -1,
          });
        }
      }
    }
  }
  const underCount = components.map(() => 0);
  for (const c of crossings) underCount[c.underComponent] += 1;
  return { components: components.length, crossings, underCount };
}

// The Wirtinger relation at one crossing. Arcs are numbered
    component by component:
// arc base[c] + k is the k-th arc of component c, from one under-
    passage to the next.
export interface Relation {
  overArc: number;
```

```
  inArc: number;
  outArc: number;
  sign: 1 | -1;
}

export function relationsOf(diagram: Diagram): Relation[] {
  const base: number[] = [];
  let total = 0;
  for (const u of diagram.underCount) {
    base.push(total);
    total += u;
  }
  // The rank of each crossing's under-passage among the under-
    passages of its component.
  const rank = new Array<number>(diagram.crossings.length).fill(0);
  for (let c = 0; c < diagram.components; c++) {
    diagram.crossings
      .map((x, index) => ({ index, position: x.underSegment + x.
    underAt, component: x.underComponent }))
      .filter((x) => x.component === c)
      .sort((p, q) => p.position - q.position)
      .forEach((x, k) => {
        rank[x.index] = k;
      });
  }
  // The arc of component c that holds a position: the last under-
    passage at or before it.
  const arcAt = (c: number, position: number): number => {
    const u = diagram.underCount[c];
    if (u === 0) throw new Error('a component with no under-passage
    is split from the rest');
    const passed = diagram.crossings.filter(
      (x) => x.underComponent === c && x.underSegment + x.underAt <=
     position,
    ).length;
    return base[c] + ((passed - 1 + u) % u);
  };
  return diagram.crossings.map((x, index) => {
    const u = diagram.underCount[x.underComponent];
    const k = rank[index];
    const start = base[x.underComponent];
    return {
      overArc: arcAt(x.overComponent, x.overSegment + x.overAt),
      inArc: start + ((k - 1 + u) % u),
      outArc: start + (k % u),
      sign: x.sign,
    };
  });
}

// The Fox colouring matrix: one row per crossing, one column per
    arc. Each row sums to zero.
export function foxMatrix(diagram: Diagram): number[][] {
  const arcs = diagram.crossings.length; // every arc starts at an
    under-passage
```

```
  const matrix = diagram.crossings.map(() => new Array<number>(arcs)
    .fill(0));
  relationsOf(diagram).forEach((rel, row) => {
    matrix[row][rel.inArc] -= 1;
    matrix[row][rel.outArc] -= 1;
    matrix[row][rel.overArc] += 2;
  });
  return matrix;
}
```

□

32 · used in 68c

7.3 Determinant

The determinant of a knot is the absolute value of any one minor of the Fox matrix, the matrix with one row and one column removed. The integer determinant is computed exactly with the Bareiss method, which divides only where the result is known to be whole. A rope with no crossings has the empty matrix, whose determinant is one, which is the unknot's value.

⟨ *Topology: determinant* 35 ⟩ ≡

```
import { foxMatrix, type Diagram } from './crossings';

// Exact integer determinant by the Bareiss method. The empty matrix
     has determinant one.
export function bareiss(input: readonly (readonly number[])[]):
    number {
  const m = input.map((row) => [...row]);
  const n = m.length;
  let sign = 1;
  let previous = 1;
  for (let k = 0; k < n - 1; k++) {
    if (m[k][k] === 0) {
      const swap = m.findIndex((row, i) => i > k && row[k] !== 0);
      if (swap === -1) return 0;
      [m[k], m[swap]] = [m[swap], m[k]];
      sign = -sign;
    }
    for (let i = k + 1; i < n; i++) {
      for (let j = k + 1; j < n; j++) {
        m[i][j] = (m[i][j] * m[k][k] - m[i][k] * m[k][j]) / previous
    ;
      }
    }
    previous = m[k][k];
  }
  return n === 0 ? 1 : sign * m[n - 1][n - 1];
}
```

```
// The determinant of a diagram of knots or links: the absolute
    value of the Fox matrix
// with its last row and column removed. A link with a component
    that is never under
// another is split, and its determinant is zero.
export function linkDeterminant(diagram: Diagram): number {
  if (diagram.components > 1 && diagram.underCount.some((u) => u ===
     0)) return 0;
  const matrix = foxMatrix(diagram);
  const k = matrix.length;
  const minor = matrix.slice(0, k - 1).map((row) => row.slice(0, k -
     1));
  return Math.abs(bareiss(minor));
}
```

□

35 · used in 68d

7.4 Recogniser

The recogniser closes the rope, builds the Fox matrix, takes the determinant, and looks the value up in the library. The library has two entries in stage one. A determinant that matches no entry gives no name, which is the honest answer until the library grows. It does not guess.

The recogniser reads a snapshot and changes nothing. It is a pure function, so the page can call it whenever the rope has moved.

⟨ *Topology: recogniser* 36 ⟩ ≡

```
import type { RopeSnapshot, Vec3 } from '../rope/snapshot';
import { alexanderOf, samePolynomial } from './alexander';
import { diagramOf } from './crossings';
import { linkDeterminant } from './determinant';
import { LIBRARY, LINKS } from './library';

export interface AnchorResult {
  name: string | null;
  determinant: number;
}

export interface KnotResult {
  name: string | null;
  determinant: number;
  alexander: number[];
  crossings: number;
  anchors: AnchorResult[];
}

// Recognises the rope's knot, and each anchor's link with the rope.
     The rope is closed by a
// straight segment from its last link to its first. That segment is
     implicit in the loop.
```

```
export function recognise(snapshot: RopeSnapshot, anchors: readonly
    (readonly Vec3[])[] = []): KnotResult {
  const rope = diagramOf([snapshot.points]);
  const determinant = linkDeterminant(rope);
  const alexander = alexanderOf(rope);
  const known = LIBRARY.find((e) => e.determinant === determinant &&
     samePolynomial(e.alexander, alexander));
  const linked = anchors.map((shape) => {
    const d = linkDeterminant(diagramOf([snapshot.points, shape]));
    const hit = d === 0 ? undefined : LINKS.find((e) => e.
    determinant === d);
    return { name: hit?.name ?? null, determinant: d };
  });
  return {
    name: known?.name ?? null,
    determinant,
    alexander,
    crossings: rope.crossings.length,
    anchors: linked,
  };
}
```

□

36 · used in 68e

7.5 Tests

The tests check the algorithm on shapes whose answers are known. A straight rope and a rope with one curl are unknots. The standard trefoil curve is a trefoil, and so is its mirror image, because the determinant cannot tell a knot from its mirror. The Fox matrix of a two-by-two case gives a determinant of three, which checks the exact arithmetic itself.

⟨ *Topology: tests* 37 ⟩ ≡

```
import { describe, expect, it } from 'vitest';
import type { Vec3 } from '../rope/snapshot';
import { bareiss } from './determinant';
import { recognise } from './recogniser';

const SAMPLES = 120;

// Sample a closed curve at evenly spaced parameter values.
function sample(curve: (t: number) => Vec3): Vec3[] {
  return Array.from({ length: SAMPLES }, (_, i) => curve((2 * Math.
    PI * i) / SAMPLES));
}

const trefoil = (t: number): Vec3 => ({
  x: Math.sin(t) + 2 * Math.sin(2 * t),
  y: Math.cos(t) - 2 * Math.cos(2 * t),
  z: -Math.sin(3 * t),
```

```
});

const mirroredTrefoil = (t: number): Vec3 => ({ ...trefoil(t), z: -
    trefoil(t).z });

// A limacon, which crosses itself once in the view. Its depth makes
     one strand pass over the other.
const curl = (t: number): Vec3 => {
  const r = 0.5 + Math.cos(t);
  return { x: r * Math.cos(t), y: r * Math.sin(t), z: 0.2 * Math.sin
    (t) };
};

// Turn a point about the x axis by a, then the y axis by b, then
    the z axis by c.
function turn(p: Vec3, a: number, b: number, c: number): Vec3 {
  const y1 = p.y * Math.cos(a) - p.z * Math.sin(a);
  const z1 = p.y * Math.sin(a) + p.z * Math.cos(a);
  const x2 = p.x * Math.cos(b) + z1 * Math.sin(b);
  const z2 = -p.x * Math.sin(b) + z1 * Math.cos(b);
  return { x: x2 * Math.cos(c) - y1 * Math.sin(c), y: x2 * Math.sin(
    c) + y1 * Math.cos(c), z: z2 };
}

describe('determinant', () => {
  it('computes exact determinants of small integer matrices', () =>
    {
    expect(bareiss([[2, -1], [-1, 2]])).toBe(3);
    expect(bareiss([[1, 0, 0], [0, 1, 0], [0, 0, 1]])).toBe(1);
    expect(bareiss([[0, 1], [1, 0]])).toBe(-1);
    expect(bareiss([])).toBe(1);
  });
});

describe('recogniser', () => {
  it('names a straight rope as the unknot', () => {
    const points = Array.from({ length: 40 }, (_, i) => ({ x: i *
    0.05, y: 0, z: 0 }));
    const result = recognise({ points, radius: 0.02 });
    expect(result.name).toBe('unknot');
    expect(result.crossings).toBe(0);
  });

  it('names a rope with one curl as the unknot', () => {
    const result = recognise({ points: sample(curl), radius: 0.02 })
    ;
    expect(result.name).toBe('unknot');
    expect(result.determinant).toBe(1);
  });

  it('names the trefoil curve as the trefoil', () => {
    const result = recognise({ points: sample(trefoil), radius: 0.02
     });
    expect(result.name).toBe('trefoil');
    expect(result.determinant).toBe(3);
```

```
    expect(result.crossings).toBe(3);
  });

  it('names the trefoil the same whichever way the rope is turned',
    () => {
    // A fixed seed, so the same 100 turns run every time.
    let seed = 11;
    const next = () => {
      seed = (seed * 16807) % 2147483647;
      return (seed / 2147483647) * Math.PI * 2;
    };
    for (let k = 0; k < 100; k++) {
      // One set of angles for the whole curve, so it is turned
    rigidly.
      const a = next();
      const b = next();
      const c = next();
      const turned = sample(trefoil).map((p) => turn(p, a, b, c));
      expect(recognise({ points: turned, radius: 0.02 }).name).toBe
    ('trefoil');
    }
  });

  it('names the mirror image of the trefoil as the trefoil', () => {
    const result = recognise({ points: sample(mirroredTrefoil),
    radius: 0.02 });
    expect(result.name).toBe('trefoil');
  });
});
```

□

37 · used in 68f

Chapter 8

Renderer

The rope is drawn as one continuous tube, not as a row of spheres or a line. The tube follows a smooth curve through the link centres, so it reads as a rope. The renderer owns no physics. It reads a snapshot and writes geometry.

The tube's vertex buffer is allocated once, when the renderer is built. Each frame the same buffer is rewritten in place, so the frame loop allocates no new geometry. That follows the three.js manual, which says a geometry is not freed automatically and that a buffer should be reused when its layout is fixed. Three.js itself allocates small temporaries when it evaluates the curve, but it does not reallocate the buffer.

8.1 The tube

Each frame the link centres become the control points of a Catmull-Rom curve. Along the curve the renderer places a ring of vertices, perpendicular to the tangent. The ring's orientation is carried from one ring to the next by projecting the previous normal onto the new tangent plane. This keeps the tube from twisting as the curve bends. The first ring starts from an arbitrary perpendicular.

The buffer's layout depends only on the link count and the radial resolution, both fixed at construction. The index buffer is built once and never changes.

⟨ *Renderer: rope tube* 40 ⟩ ≡

```
import * as THREE from 'three';
import type { RopeSnapshot } from '../rope/snapshot';

const RADIAL = 8;
const SAMPLES_PER_LINK = 4;
```

```
// One tube along the rope. The buffers are allocated once in the
    constructor, and
// update() rewrites the vertex positions in place.
export class RopeTube {
  readonly mesh: THREE.Mesh;
  readonly geometry = new THREE.BufferGeometry();
  private readonly material = new THREE.MeshStandardMaterial({
    color: 0xd9a441,
    roughness: 0.6,
    side: THREE.DoubleSide,
  });
  private readonly positions: Float32Array;
  private readonly attribute: THREE.BufferAttribute;
  private readonly controls: THREE.Vector3[];
  private readonly curve: THREE.CatmullRomCurve3;
  private readonly rings: number;
  private readonly radius: number;
  private readonly center = new THREE.Vector3();
  private readonly tangent = new THREE.Vector3();
  private readonly normal = new THREE.Vector3();
  private readonly binormal = new THREE.Vector3();
  private readonly seed = new THREE.Vector3();

  constructor(linkCount: number, radius: number) {
    this.radius = radius;
    this.rings = (linkCount - 1) * SAMPLES_PER_LINK + 1;

    this.positions = new Float32Array(this.rings * (RADIAL + 1) * 3)
    ;
    this.attribute = new THREE.BufferAttribute(this.positions, 3);
    this.geometry.setAttribute('position', this.attribute);

    const indices: number[] = [];
    for (let r = 0; r < this.rings - 1; r++) {
      for (let s = 0; s < RADIAL; s++) {
        const a = r * (RADIAL + 1) + s;
        const c = a + (RADIAL + 1);
        indices.push(a, c, a + 1, a + 1, c, c + 1);
      }
    }
    this.geometry.setIndex(indices);

    this.controls = Array.from({ length: linkCount }, () => new
    THREE.Vector3());
    this.curve = new THREE.CatmullRomCurve3(this.controls);
    this.mesh = new THREE.Mesh(this.geometry, this.material);
    // The bounds are not recomputed each frame, so culling would
    use stale ones.
    this.mesh.frustumCulled = false;
  }

  update(snapshot: RopeSnapshot): void {
    snapshot.points.forEach((p, i) => this.controls[i].set(p.x, p.y,
     p.z));
    this.curve.points = this.controls;
```

```
    for (let r = 0; r < this.rings; r++) {
      const t = r / (this.rings - 1);
      this.curve.getPoint(t, this.center);
      this.curve.getTangent(t, this.tangent);
      if (r === 0) {
        // Start from any direction that is not along the tangent.
        this.seed.set(Math.abs(this.tangent.x) > 0.9 ? 0 : 1, Math.
    abs(this.tangent.x) > 0.9 ? 1 : 0, 0);
        this.normal.copy(this.seed);
      }
      // Project the carried normal onto the plane perpendicular to
    the tangent.
      this.normal.addScaledVector(this.tangent, -this.normal.dot(
    this.tangent)).normalize();
      this.binormal.crossVectors(this.tangent, this.normal);

      for (let s = 0; s <= RADIAL; s++) {
        const angle = (s / RADIAL) * Math.PI * 2;
        const cos = Math.cos(angle) * this.radius;
        const sin = Math.sin(angle) * this.radius;
        const o = (r * (RADIAL + 1) + s) * 3;
        this.positions[o] = this.center.x + this.normal.x * cos +
    this.binormal.x * sin;
        this.positions[o + 1] = this.center.y + this.normal.y * cos
    + this.binormal.y * sin;
        this.positions[o + 2] = this.center.z + this.normal.z * cos
    + this.binormal.z * sin;
      }
    }
    this.attribute.needsUpdate = true;
  }

  dispose(): void {
    this.geometry.dispose();
    this.material.dispose();
  }
}
```

□

40 · used in 70a

8.2 Tests

The renderer is tested without a browser. A straight rope is the simplest case: every vertex must lie exactly one radius from the rope's axis. The test also checks that the buffer is the same object before and after an update, which is the no-allocation promise made above.

⟨ *Renderer: tube tests* 42 ⟩ ≡

```
import { describe, expect, it } from 'vitest';
```

```
import type { RopeSnapshot } from '../rope/snapshot';
import { RopeTube } from './ropeTube';

const LINKS = 40;
const RADIUS = 0.02;

const straight: RopeSnapshot = {
  radius: RADIUS,
  points: Array.from({ length: LINKS }, (_, i) => ({ x: 0, y: -i *
    0.05, z: 0 })),
};

describe('rope tube', () => {
  it('keeps the same vertex buffer across updates', () => {
    const tube = new RopeTube(LINKS, RADIUS);
    const before = tube.geometry.getAttribute('position').array;
    tube.update(straight);
    tube.update(straight);
    expect(tube.geometry.getAttribute('position').array).toBe(before
    );
    tube.dispose();
  });

  it('places every vertex one radius from the rope axis', () => {
    const tube = new RopeTube(LINKS, RADIUS);
    tube.update(straight);
    const array = tube.geometry.getAttribute('position').array;
    for (let i = 0; i < array.length; i += 3) {
      const distance = Math.hypot(array[i], array[i + 2]);
      expect(distance).toBeCloseTo(RADIUS, 6);
    }
    tube.dispose();
  });
});
```

□

42 · used in 70b

Chapter 9

Lessons and the page

The page is React and Tailwind. It holds the lesson text, the recognised name and the gravity switch, and it does not hold any per-frame state. The scene, with the physics world, the renderer and the rope tube, lives in one component, which creates them when it mounts and disposes of them when it unmounts.

9.1 Lesson record

A lesson is data. It names the knot, the stage that teaches it, its web sources, the invariants a recogniser must match, and the prompt the player reads. Stage one has one record, the overhand knot, so the page has something to show. The guided, sandbox and quiz modes arrive in later stages and read the same records.

The source is the Knot Atlas entry for 3_1, which the research chapter records as confirmed. The determinant is three, as the topology chapter computes. The steps are written for this page and are not from a cited tying guide, so stage six must replace them with cited ones.

⟨ *Lessons: record and stub* 44 ⟩ ≡

⟨ *Lessons: guided steps* 62 ⟩

```
export interface Invariants {
  determinant: number;
}

export interface LessonRecord {
  id: string;
  name: string;
  stage: number;
  sources: readonly string[];
  invariants: Invariants;
  prompt: string;
```

```
  steps: readonly LessonStep[];
}

export const STAGE_ONE_LESSONS: readonly LessonRecord[] = [
  {
    id: 'overhand',
    name: 'Overhand knot',
    stage: 1,
    sources: ['https://katlas.org/wiki/3_1'],
    invariants: { determinant: 3 },
    prompt: 'Tie an overhand knot in the rope. Drag a link to move
    it. Scroll to move it toward or away from you. Drag empty space
    to orbit.',
    steps: [
      { text: 'Make a loop in the rope, so that one part crosses
    over another.', check: { kind: 'crossings', atLeast: 1 } },
      { text: 'Take one end over the rope, then back under it.',
    check: { kind: 'crossings', atLeast: 2 } },
      { text: 'Pass the same end through the loop you made.', check:
     { kind: 'crossings', atLeast: 3 } },
      { text: 'Draw the ends apart slowly and let the rope settle.
    The name appears when the knot is tied.', check: { kind: 'knot',
     name: 'trefoil' } },
    ],
  },
];
```

□

44 · used in 67e

9.2 Scene objects

The bar and the floor are scene data: boxes with a centre and half sizes. The rope model receives them as plain shapes, and the renderer draws them as meshes. Neither owns them. The bar lies along the depth axis, above the rope, so a rope dragged up to it can drape over it, and it cannot pass through it. The floor is where the rope falls when gravity is on.

⟨ *Anchors: bar* 45 ⟩ ≡

```
import type { Box } from '../rope/snapshot';

export const BAR: Box = {
  center: { x: 0.6, y: 0.45, z: 0 },
  half: { x: 0.02, y: 0.02, z: 0.4 },
};
```

□

45 · used in 67f

9.3 Rope stage

The stage component owns the whole scene. Four things keep it honest.

The physics loop runs only while the rope moves. When the fastest link is slower than the settle speed for thirty frames, the loop stops, and a pointer event, a wheel turn or a gravity change starts it again. That is render on demand, as the architecture chapter asks.

Recognition runs at most four times a second while the rope moves, and once when it settles. It sets state only when the name changes, so React re-renders on a change of result, never on each frame.

Depth is the distance from the camera measured straight ahead, along the view direction. A click grabs the nearest link under the pointer, within a few pixels, and records that link's depth. While the button is held, the held point is where the mouse ray meets the plane at that depth. It stays under the cursor, and it keeps its depth as the cursor moves. The wheel changes the depth: up brings the point closer to the camera, down pushes it farther. Measuring the distance along the ray instead would make the point drift into the scene as the cursor moved away from the middle of the view.

Gravity is a switch in the page. The page keeps the choice in a ref, so the scene can read it when it finishes loading, and it updates the rope when the switch is pressed.

The camera orbits when the player drags on empty space, and the wheel zooms it. Over a link, the same actions move the link instead, so the camera is locked while a link is held. The reset button returns the camera to its start view. Orbiting changes only what the player sees. Recognition reads the rope along the same fixed axis whatever the camera does, so the answer does not change with the view.

⟨ *UI: rope stage* 46 ⟩ ≡

```
import { useEffect, useRef, useState } from 'react';
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.
    js';
import { RopeTube } from '../render/ropeTube';
import { BAR } from '../anchors/bar';
import { LINK_COUNT, LINK_RADIUS, createRope, initPhysics, type Rope
     } from '../rope/rope';
import type { Box } from '../rope/snapshot';
import { recognise, type KnotResult } from '../topology/recogniser';

const FLOOR: Box = { center: { x: 0, y: -1, z: 0 }, half: { x: 1.5,
    y: 0.05, z: 1.5 } };
const ROPE_START = { x: -0.975, y: 0.2, z: 0 };
const SETTLE_SPEED = 0.01;
const SETTLE_FRAMES = 30;
const RECOGNISE_EVERY_MS = 250;
const DEPTH_STEP = 0.0003;
```

```
const NEAR = 0.3;
const FAR = 6;
const PICK_RADIUS = 36;

export function RopeStage({ onRecognise }: { onRecognise: (result:
    KnotResult) => void }) {
  const hostRef = useRef<HTMLDivElement>(null);
  const report = useRef(onRecognise);
  report.current = onRecognise;
  const gravityRef = useRef(false);
  const controls = useRef<{ setGravity(on: boolean): void } | null>(
    null);
  const view = useRef<{ reset(): void } | null>(null);
  const [name, setName] = useState<string | null>(null);
  const [gravityOn, setGravityOn] = useState(false);

  const resetView = () => {
    view.current?.reset();
  };

  const toggleGravity = () => {
    gravityRef.current = !gravityRef.current;
    setGravityOn(gravityRef.current);
    controls.current?.setGravity(gravityRef.current);
  };

  useEffect(() => {
    const host = hostRef.current;
    if (!host) return;

    let disposed = false;
    let frame = 0;
    let running = false;
    let still = 0;
    let lastRecognised = 0;
    let cleanup = () => {};
    const owned: { dispose(): void }[] = [];

    const raycaster = new THREE.Raycaster();
    const pointer = new THREE.Vector2();
    const hit = new THREE.Vector3();
    const screen = new THREE.Vector3();
    const facing = new THREE.Vector3();
    let grabbing = false;
    let depth = 0;

    initPhysics().then(() => {
      if (disposed) return;

      const rope: Rope = createRope(ROPE_START, [BAR, FLOOR]);
      rope.setGravity(gravityRef.current);
      const tube = new RopeTube(LINK_COUNT, LINK_RADIUS);
      owned.push(rope, tube);

      const scene = new THREE.Scene();
```

```
  scene.background = new THREE.Color(0x0f172a);
  scene.add(tube.mesh);
  scene.add(new THREE.AmbientLight(0xffffff, 0.6));
  const light = new THREE.DirectionalLight(0xffffff, 1.2);
  light.position.set(1, 2, 3);
  scene.add(light);

  const addBox = (box: Box, color: number) => {
    const geometry = new THREE.BoxGeometry(box.half.x * 2, box.
half.y * 2, box.half.z * 2);
    const material = new THREE.MeshStandardMaterial({ color });
    const mesh = new THREE.Mesh(geometry, material);
    mesh.position.set(box.center.x, box.center.y, box.center.z);
    scene.add(mesh);
    owned.push(geometry, material);
  };
  addBox(BAR, 0x64748b);
  addBox(FLOOR, 0x1e293b);

  const camera = new THREE.PerspectiveCamera(40, host.
clientWidth / host.clientHeight, 0.01, 100);
  camera.position.set(1.1, 0.9, 3.4);
  camera.lookAt(0.1, 0.1, 0);

  const renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
  renderer.setSize(host.clientWidth, host.clientHeight);
  host.appendChild(renderer.domElement);

  const canvas = renderer.domElement;

  // Empty space orbits the camera, and the wheel zooms. A held
link turns both off.
  const orbit = new OrbitControls(camera, canvas);
  orbit.enableDamping = true;
  orbit.minDistance = 1;
  orbit.maxDistance = 8;
  orbit.target.set(0.1, 0.1, 0);
  orbit.update();
  orbit.saveState();
  owned.push(orbit);
  view.current = {
    reset() {
      orbit.reset();
      wake();
    },
  };

  controls.current = {
    setGravity(on) {
      rope.setGravity(on);
      wake();
    },
  };
```

```
  const animate = () => {
    orbit.update();
    const fastest = rope.step();
    const snapshot = rope.snapshot();
    tube.update(snapshot);
    renderer.render(scene, camera);

    still = fastest < SETTLE_SPEED ? still + 1 : 0;
    const now = performance.now();
    if (still === SETTLE_FRAMES || (still === 0 && now -
lastRecognised > RECOGNISE_EVERY_MS)) {
      lastRecognised = now;
      const result = recognise(snapshot);
      setName(result.name);
      report.current(result);
    }
    if (still < SETTLE_FRAMES) {
      frame = requestAnimationFrame(animate);
    } else {
      running = false;
    }
  };

  const wake = () => {
    still = 0;
    if (!running) {
      running = true;
      frame = requestAnimationFrame(animate);
    }
  };

  // Orbiting, damping and zoom redraw the scene, so they wake
the loop.
  orbit.addEventListener('change', wake);

  const pointerRay = (event: PointerEvent) => {
    const rect = canvas.getBoundingClientRect();
    pointer.set(
      ((event.clientX - rect.left) / rect.width) * 2 - 1,
      -((event.clientY - rect.top) / rect.height) * 2 + 1,
    );
    raycaster.setFromCamera(pointer, camera);
  };

  // The nearest link whose on-screen position is within
PICK_RADIUS pixels, or null.
  const pickLink = (event: PointerEvent): number | null => {
    const rect = canvas.getBoundingClientRect();
    const points = rope.snapshot().points;
    let best: number | null = null;
    let bestDistance = PICK_RADIUS;
    for (let i = 0; i < points.length; i++) {
      const p = points[i];
      screen.set(p.x, p.y, p.z).project(camera);
      const x = rect.left + ((screen.x + 1) / 2) * rect.width;
```

```
      const y = rect.top + ((1 - screen.y) / 2) * rect.height;
      const distance = Math.hypot(x - event.clientX, y - event.
clientY);
      if (distance < bestDistance) {
        bestDistance = distance;
        best = i;
      }
    }
    return best;
  };

  // The point on the mouse ray that lies the given distance
ahead of the camera, along the view direction.
  const pointAtDepth = (distance: number): THREE.Vector3 => {
    camera.getWorldDirection(facing);
    const along = raycaster.ray.direction.dot(facing);
    return raycaster.ray.at(distance / along, hit);
  };

  const onDown = (event: PointerEvent) => {
    const index = pickLink(event);
    if (index === null) return;
    pointerRay(event);
    const at = rope.snapshot().points[index];
    // The held point keeps this depth until the wheel changes
it.
    camera.getWorldDirection(facing);
    depth = hit.set(at.x, at.y, at.z).sub(camera.position).dot(
facing);
    grabbing = true;
    orbit.enabled = false;
    canvas.setPointerCapture(event.pointerId);
    rope.grabLink(index, at);
    wake();
  };
  const onMove = (event: PointerEvent) => {
    if (!grabbing) return;
    pointerRay(event);
    rope.moveGrab(pointAtDepth(depth));
    wake();
  };
  const onUp = () => {
    if (!grabbing) return;
    grabbing = false;
    orbit.enabled = true;
    rope.releaseGrab();
    wake();
  };
  const onWheel = (event: WheelEvent) => {
    if (!grabbing) return;
    // The ray is the last pointer ray, so the held point stays
under the cursor.
    depth = THREE.MathUtils.clamp(depth + event.deltaY *
DEPTH_STEP, NEAR, FAR);
    rope.moveGrab(pointAtDepth(depth));
```

```
      wake();
    };
    const onResize = () => {
      camera.aspect = host.clientWidth / host.clientHeight;
      camera.updateProjectionMatrix();
      renderer.setSize(host.clientWidth, host.clientHeight);
    };

    // Capture phase, so a click on a link is taken before the
  orbit controls see it.
    canvas.addEventListener('pointerdown', onDown, { capture: true
   });
    canvas.addEventListener('pointermove', onMove);
    canvas.addEventListener('pointerup', onUp);
    canvas.addEventListener('pointercancel', onUp);
    canvas.addEventListener('wheel', onWheel);
    window.addEventListener('resize', onResize);

    cleanup = () => {
      canvas.removeEventListener('pointerdown', onDown, { capture:
   true });
      canvas.removeEventListener('pointermove', onMove);
      canvas.removeEventListener('pointerup', onUp);
      canvas.removeEventListener('pointercancel', onUp);
      canvas.removeEventListener('wheel', onWheel);
      window.removeEventListener('resize', onResize);
      renderer.dispose();
      renderer.domElement.remove();
    };

    wake();
  });

  return () => {
    disposed = true;
    cancelAnimationFrame(frame);
    cleanup();
    controls.current = null;
    for (const item of owned) item.dispose();
  };
}, []);

return (
  <div className="relative h-full w-full">
    <div ref={hostRef} className="absolute inset-0" />
    <p className="pointer-events-none absolute bottom-3 left-3
  rounded bg-slate-900/80 px-2 py-1 text-sm text-slate-200">
      Recognised: {name ?? 'nothing yet'}
    </p>
    <div className="absolute bottom-3 right-3 flex gap-2">
      <button
        type="button"
        onClick={resetView}
        className="rounded bg-slate-900/80 px-3 py-1 text-sm text-
  slate-100 hover:bg-slate-800"
```

```
        >
          Reset view
        </button>
        <button
          type="button"
          onClick={toggleGravity}
          className="rounded bg-slate-900/80 px-3 py-1 text-sm text-
    slate-100 hover:bg-slate-800"
        >
          Gravity: {gravityOn ? 'on' : 'off'}
        </button>
      </div>
    </div>
  );
}
```

□

46 · used in 67d

9.4 Application

The application is one screen: the lesson's name, its prompt and its numbered steps at the top, the rope stage filling the rest. The steps are what the player follows, so a lesson cannot be used without them. The stage is the only part with per-frame work.

⟨ *UI: application* 52 ⟩ ≡

```
import { useCallback, useState } from 'react';
import { STAGE_ONE_LESSONS } from './lessons/record';
import type { KnotResult } from './topology/recogniser';
import { GuidedPanel } from './ui/GuidedPanel';
import { RopeStage } from './ui/RopeStage';

export function App() {
  const lesson = STAGE_ONE_LESSONS[0];
  const [result, setResult] = useState<KnotResult | null>(null);
  // Recognition runs several times a second. Only a change of name
    or crossings re-renders the page.
  const report = useCallback((next: KnotResult) => {
    setResult((prev) => (prev && prev.name === next.name && prev.
    crossings === next.crossings ? prev : next));
  }, []);
  return (
    <main className="flex h-full flex-col bg-slate-950 text-slate
    -100">
      <header className="border-b border-slate-800 px-4 py-3">
        <h1 className="text-lg font-semibold">Roppi</h1>
        <p className="text-sm text-slate-400">
          {lesson.name}. {lesson.prompt}
        </p>
        <GuidedPanel steps={lesson.steps} result={result} />
```

```
      </header>
      <section className="relative min-h-0 flex-1">
        <RopeStage onRecognise={report} />
      </section>
    </main>
  );
}
```

□

52 · used in 67c

Chapter 10

Recogniser, stage two

Stage two widens the recogniser from two knots to the taught set, and from one rope to a rope tied to anchors. It adds the Alexander polynomial, which separates knots that share a determinant. It adds anchors as closed loops, so a rope hitched on an anchor is recognised as a link. And a test checks at build time that no two library entries share both invariants.

The two open decisions were settled as recommended. A link is recognised by its determinant. Anchors are stored as closed polylines for recognition, and the physics shapes stay separate.

10.1 Alexander polynomial

The determinant alone cannot separate every taught knot. The Alexander polynomial is the second invariant. It comes from the same crossing data as the Fox matrix, through the Wirtinger relations. At a positive crossing with over arc o, incoming under arc i and outgoing under arc j, the Alexander row has $1 - t$ at o, t at i and -1 at j. A negative crossing gives $t - 1$, 1 and $-t$, which is the same relation multiplied by a unit. The determinant of the matrix with one row and one column removed is the polynomial, up to a unit $\pm t^k$.

The polynomial is found by evaluating that determinant at k integer points and interpolating. The determinant at each point is an exact integer, so the interpolation is exact. The result is normalised: its lowest power is zero, and its first coefficient is positive. Two knots match only when the determinant and the normalised polynomial both agree. Mirror images agree on both, so they still match.

⟨ *Recogniser: Alexander polynomial* 54 ⟩ ≡

```
import type { Diagram } from './crossings';
import { relationsOf } from './crossings';
```

```
// A polynomial in t, lowest power first.
type Poly = number[];

function addPoly(a: Poly, b: Poly): Poly {
  const n = Math.max(a.length, b.length);
  return Array.from({ length: n }, (_, i) => (a[i] ?? 0) + (b[i] ??
    0));
}

// The Alexander matrix: one row per crossing, one column per arc,
    with polynomial entries.
function alexanderMatrix(diagram: Diagram): Poly[][] {
  const k = diagram.crossings.length;
  const matrix: Poly[][] = Array.from({ length: k }, () => Array.
    from({ length: k }, (): Poly => []));
  relationsOf(diagram).forEach((rel, row) => {
    const entries: [number, Poly][] =
      rel.sign === 1
        ? [
            [rel.overArc, [1, -1]],
            [rel.inArc, [0, 1]],
            [rel.outArc, [-1]],
          ]
        : [
            [rel.overArc, [-1, 1]],
            [rel.inArc, [1]],
            [rel.outArc, [0, -1]],
          ];
    for (const [col, poly] of entries) matrix[row][col] = addPoly(
    matrix[row][col], poly);
  });
  return matrix;
}

function evalAt(p: Poly, x: bigint): bigint {
  let v = 0n;
  for (let i = p.length - 1; i >= 0; i--) v = v * x + BigInt(p[i]);
  return v;
}

// Exact determinant of an integer matrix, by the Bareiss method.
function bareissBig(input: bigint[][]): bigint {
  const m = input.map((row) => [...row]);
  const n = m.length;
  let sign = 1n;
  let previous = 1n;
  for (let k = 0; k < n - 1; k++) {
    if (m[k][k] === 0n) {
      const swap = m.findIndex((row, i) => i > k && row[k] !== 0n);
      if (swap === -1) return 0n;
      [m[k], m[swap]] = [m[swap], m[k]];
      sign = -sign;
    }
    for (let i = k + 1; i < n; i++) {
      for (let j = k + 1; j < n; j++) {
```

```
        m[i][j] = (m[i][j] * m[k][k] - m[i][k] * m[k][j]) / previous
    ;
      }
    }
    previous = m[k][k];
  }
  return n === 0 ? 1n : sign * m[n - 1][n - 1];
}

// Multiplies a polynomial by (t + c).
function timesLinear(poly: bigint[], c: bigint): bigint[] {
  const out = new Array<bigint>(poly.length + 1).fill(0n);
  poly.forEach((v, m) => {
    out[m + 1] += v;
    out[m] += v * c;
  });
  return out;
}

// The polynomial with integer coefficients through the points (xs[i
    ], ys[i]).
function interpolate(xs: bigint[], ys: bigint[]): bigint[] {
  const denom = xs.map((xi, i) => xs.reduce((acc, xj, j) => (j === i
     ? acc : acc * (xi - xj)), 1n));
  const common = denom.reduce((acc, d) => acc * d, 1n);
  const total = new Array<bigint>(xs.length).fill(0n);
  xs.forEach((_, i) => {
    let numerator: bigint[] = [1n];
    xs.forEach((xj, j) => {
      if (j !== i) numerator = timesLinear(numerator, -xj);
    });
    const weight = (common / denom[i]) * ys[i];
    numerator.forEach((c, m) => {
      total[m] += c * weight;
    });
  });
  return total.map((c) => c / common);
}

// Lowest power zero, and the first coefficient positive.
function normalise(p: number[]): number[] {
  let lo = 0;
  while (lo < p.length && p[lo] === 0) lo++;
  let hi = p.length;
  while (hi > lo && p[hi - 1] === 0) hi--;
  const out = p.slice(lo, hi);
  if (out.length === 0) return [0];
  return out[0] < 0 ? out.map((c) => -c) : out;
}

// The Alexander polynomial of a knot diagram, normalised.
export function alexanderOf(diagram: Diagram): number[] {
  const k = diagram.crossings.length;
  if (k === 0) return [1];
  const matrix = alexanderMatrix(diagram);
```

```
  const minor = matrix.slice(0, k - 1).map((row) => row.slice(0, k -
    1));
  // The minor's determinant has degree at most k - 1, so k sample
    points fix it.
  const xs = Array.from({ length: k }, (_, i) => BigInt(i + 2));
  const ys = xs.map((x) => bareissBig(minor.map((row) => row.map((
    entry) => evalAt(entry, x)))));
  return normalise(interpolate(xs, ys).map((c) => Number(c)));
}

export function samePolynomial(a: readonly number[], b: readonly
    number[]): boolean {
  return a.length === b.length && a.every((c, i) => c === b[i]);
}
```

□

54 · used in 68g

10.2 Taught library

The library is the set of knots the lessons teach, and of links a rope makes with an anchor. Each knot entry has a name, a determinant and a normalised Alexander polynomial. The values are those of the knots themselves, and the tests below compute them from scripted ropes, so a wrong value shows as a failing test.

The stage-two knots are the unknot, the trefoil (the overhand) and the figure-eight. The figure-eight is the knot of the research's table for the figure-eight on a bight. The anchoring, belaying and rappel lessons add more in stage six, and each addition must pass the collision check below.

The link entry is the hitch. A rope making one turn around an anchor forms a link with determinant two.

⟨ *Recogniser: taught library* 57 ⟩ ≡

```
export interface KnotEntry {
  name: string;
  determinant: number;
  alexander: readonly number[];
}

export const LIBRARY: readonly KnotEntry[] = [
  { name: 'unknot', determinant: 1, alexander: [1] },
  { name: 'trefoil', determinant: 3, alexander: [1, -1, 1] },
  { name: 'figure-eight', determinant: 5, alexander: [1, -3, 1] },
];

export interface LinkEntry {
  name: string;
  determinant: number;
}
```

```
// A rope hitched on an anchor. Determinant two is the Hopf link.
export const LINKS: readonly LinkEntry[] = [{ name: 'hitch',
    determinant: 2 }];
```

□

57 · used in 69a

10.3 Anchors as closed loops

Each anchor is a closed polyline for recognition. The last point joins the first. A bar is a loop from its two ends back through its support. A ring lies in a plane, a tree loop lies round a vertical trunk, and a carabiner is a loop with two round ends. The physics shapes of the scene stay separate. Only the recogniser sees these loops.

⟨ *Recogniser: anchors as components* 58 ⟩ ≡

```
import type { Vec3 } from '../rope/snapshot';

const SAMPLES = 48;

// A bar: the segment from a to b, then back through a support
    offset by s.
export function barLoop(a: Vec3, b: Vec3, s: Vec3): Vec3[] {
  const add = (p: Vec3, q: Vec3): Vec3 => ({ x: p.x + q.x, y: p.y +
    q.y, z: p.z + q.z });
  return [a, b, add(b, s), add(a, s)];
}

// A ring in the xy plane, round a centre.
export function ringXY(centre: Vec3, radius: number): Vec3[] {
  return Array.from({ length: SAMPLES }, (_, i) => {
    const t = (2 * Math.PI * i) / SAMPLES;
    return { x: centre.x + radius * Math.cos(t), y: centre.y +
    radius * Math.sin(t), z: centre.z };
  });
}

// A loop round a vertical trunk: a ring in the xz plane.
export function treeLoop(centre: Vec3, radius: number): Vec3[] {
  return Array.from({ length: SAMPLES }, (_, i) => {
    const t = (2 * Math.PI * i) / SAMPLES;
    return { x: centre.x + radius * Math.cos(t), y: centre.y, z:
    centre.z + radius * Math.sin(t) };
  });
}

// A carabiner's loop, seen face-on: two straight sides with round
    ends at x = plus or minus halfLength.
export function carabiner(centre: Vec3, halfLength: number, radius:
    number): Vec3[] {
```

```
  const half = SAMPLES / 2;
  const right = Array.from({ length: half }, (_, i) => {
    const t = -Math.PI / 2 + (Math.PI * i) / (half - 1);
    return { x: centre.x + halfLength + radius * Math.cos(t), y:
    centre.y + radius * Math.sin(t), z: centre.z };
  });
  const left = Array.from({ length: half }, (_, i) => {
    const t = Math.PI / 2 + (Math.PI * i) / (half - 1);
    return { x: centre.x - halfLength + radius * Math.cos(t), y:
    centre.y + radius * Math.sin(t), z: centre.z };
  });
  return [...right, ...left];
}
```

□

58 · used in 69b

10.4 Tests

The stage-two tests are the proof for the stage. Each library knot is recognised from a scripted rope of that knot. The Alexander polynomials are computed and compared with the library. A rope turned round a bar is a link with the bar, and a rope clear of the bar is not. The collision check fails if two library entries share both invariants.

The figure-eight curve is the standard parametrisation, sampled at 120 points. The hitch is one turn of rope round the bar.

⟨ *Recogniser: stage two tests* 59 ⟩ ≡

```
import { describe, expect, it } from 'vitest';
import type { Vec3 } from '../rope/snapshot';
import { barLoop, ringXY } from '../anchors/shapes';
import { alexanderOf, samePolynomial } from './alexander';
import { diagramOf } from './crossings';
import { LIBRARY, LINKS } from './library';
import { recognise } from './recogniser';

const SAMPLES = 120;

function sample(curve: (t: number) => Vec3): Vec3[] {
  return Array.from({ length: SAMPLES }, (_, i) => curve((2 * Math.
    PI * i) / SAMPLES));
}

const trefoil = (t: number): Vec3 => ({
  x: Math.sin(t) + 2 * Math.sin(2 * t),
  y: Math.cos(t) - 2 * Math.cos(2 * t),
  z: -Math.sin(3 * t),
});

const figureEight = (t: number): Vec3 => {
```

```
  const r = 2 + Math.cos(2 * t);
  return { x: r * Math.cos(3 * t), y: r * Math.sin(3 * t), z: Math.
    sin(4 * t) };
};

// The bar runs along z, and its support hangs off to the side at y
    = -0.6.
const bar = barLoop({ x: 0, y: 0, z: -0.4 }, { x: 0, y: 0, z: 0.4 },
     { x: 0, y: -0.6, z: 0 });

describe('stage two recogniser', () => {
  it('names the figure-eight from its scripted rope', () => {
    const result = recognise({ points: sample(figureEight), radius:
    0.02 });
    expect(result.name).toBe('figure-eight');
    expect(result.determinant).toBe(5);
  });

  it('gives the trefoil and the figure-eight their Alexander
    polynomials', () => {
    expect(alexanderOf(diagramOf([sample(trefoil)]))).toEqual([1,
    -1, 1]);
    expect(alexanderOf(diagramOf([sample(figureEight)]))).toEqual
    ([1, -3, 1]);
  });

  it('names a rope turned once round the bar as a hitch', () => {
    const turn = Array.from({ length: 40 }, (_, i) => {
      const t = (2 * Math.PI * i) / 40;
      return { x: 0.3 * Math.cos(t), y: 0.3 * Math.sin(t), z: 0 };
    });
    const result = recognise({ points: turn, radius: 0.02 }, [bar]);
    expect(result.anchors[0]).toEqual({ name: 'hitch', determinant:
    2 });
  });

  it('finds no link with a rope that is clear of the bar', () => {
    const clear = ringXY({ x: 2, y: 0, z: 0 }, 0.3);
    const result = recognise({ points: clear, radius: 0.02 }, [bar])
    ;
    expect(result.anchors[0]).toEqual({ name: null, determinant: 0
    });
  });

  it('keeps every library entry apart from the others', () => {
    for (let i = 0; i < LIBRARY.length; i++) {
      for (let j = i + 1; j < LIBRARY.length; j++) {
        const a = LIBRARY[i];
        const b = LIBRARY[j];
        const shared = a.determinant === b.determinant &&
    samePolynomial(a.alexander, b.alexander);
        expect(shared, `${a.name} and ${b.name} share both
    invariants`).toBe(false);
      }
    }
```

```
  });

  it('keeps every link entry named once', () => {
    const names = LINKS.map((e) => e.name);
    expect(new Set(names).size).toBe(names.length);
  });
});
```

□

59 · used in 69c

Chapter 11

Guided mode, stage three

Stage three teaches one knot step by step. The overhand knot is the first guided lesson. The page shows its steps in order, and a step is done only when the recogniser confirms it. A timer never advances a step, and a click never does.

11.1 What a step is

Each lesson step has the instruction the player reads, and a check. A check is a fact about the rope that the recogniser computes from a snapshot. The check is one of two kinds. A crossings check passes when the diagram has at least a given number of crossings. A knot check passes when the recogniser names the knot. The lesson is finished when every step passes.

⟨ *Lessons: guided steps* 62 ⟩ ≡

```
// A check is a fact about the rope that the recogniser computes.
export type StepCheck = { kind: 'crossings'; atLeast: number } | {
    kind: 'knot'; name: string };

export interface LessonStep {
  text: string;
  check: StepCheck;
}
```

□

62 · used in 44

11.2 The check for a step

The first step whose check fails is the current step. Steps are not remembered once passed. If the player undoes a crossing, the lesson goes back to the step

that needs it. That is why the crossing checks are cumulative: a rope with three crossings passes the first two.

A crossing count depends on how the rope is turned, because the diagram is read along the depth axis. The page does not correct for it. A player who turns the rope may see a crossing count change without changing the knot.

⟨ *Guided: step check* 63a ⟩ ≡

```
import type { KnotResult } from '../topology/recogniser';
import type { LessonStep, StepCheck } from './record';

export function checkPasses(check: StepCheck, result: KnotResult |
    null): boolean {
  if (!result) return false;
  if (check.kind === 'crossings') return result.crossings >= check.
    atLeast;
  return result.name === check.name;
}

// The index of the current step: the first whose check fails. Equal
     to steps.length when all pass.
export function currentStep(steps: readonly LessonStep[], result:
    KnotResult | null): number {
  const index = steps.findIndex((step) => !checkPasses(step.check,
    result));
  return index === -1 ? steps.length : index;
}
```

□

63a · used in 69d

11.3 The panel

The panel lists the steps. Steps before the current one are marked done, the current one is highlighted, and the rest are muted. When every step passes the panel says the lesson is complete. The panel receives the recogniser's result from the page, and the page updates it only when the name or the crossings change.

⟨ *UI: guided panel* 63b ⟩ ≡

```
import type { LessonStep } from '../lessons/record';
import { currentStep } from '../lessons/guided';
import type { KnotResult } from '../topology/recogniser';

export function GuidedPanel({ steps, result }: { steps: readonly
    LessonStep[]; result: KnotResult | null }) {
  const current = currentStep(steps, result);
  return (
    <div className="mt-2">
```

```
      <ol className="list-decimal space-y-0.5 pl-5 text-sm">
        {steps.map((step, index) => {
          const tone =
            index < current ? 'text-emerald-400' : index === current
    ? 'font-semibold text-slate-100' : 'text-slate-500';
          return (
            <li key={step.text} className={tone}>
              {index < current ? ' ' : ''}
              {step.text}
            </li>
          );
        })}
      </ol>
      <p className="mt-1 text-sm text-slate-300">
        {current === steps.length ? 'Lesson complete.' : `Step ${
    current + 1} of ${steps.length}.`}
      </p>
    </div>
  );
}
```

□

63b · used in 69f

11.4 Tests

The tests drive the lesson with the recogniser's own results. A straight rope has no crossings, so no step passes. The scripted trefoil passes every step. A rope that loses a crossing goes back to the step that needs it.

⟨ *Guided: tests* 64 ⟩ ≡

```
import { describe, expect, it } from 'vitest';
import type { Vec3 } from '../rope/snapshot';
import { recognise } from '../topology/recogniser';
import { STAGE_ONE_LESSONS } from './record';
import { currentStep } from './guided';

const steps = STAGE_ONE_LESSONS[0].steps;

const trefoil = (t: number): Vec3 => ({
  x: Math.sin(t) + 2 * Math.sin(2 * t),
  y: Math.cos(t) - 2 * Math.cos(2 * t),
  z: -Math.sin(3 * t),
});

const sample = (curve: (t: number) => Vec3, n: number): Vec3[] =>
  Array.from({ length: n }, (_, i) => curve((2 * Math.PI * i) / n));

describe('guided overhand lesson', () => {
  it('starts at the first step with no result', () => {
    expect(currentStep(steps, null)).toBe(0);
```

```
  });

  it('does not advance for a straight rope', () => {
    const straight = Array.from({ length: 40 }, (_, i) => ({ x: i *
    0.05, y: 0, z: 0 }));
    expect(currentStep(steps, recognise({ points: straight, radius:
    0.02 }))).toBe(0);
  });

  it('is complete for the scripted trefoil', () => {
    const tied = recognise({ points: sample(trefoil, 120), radius:
    0.02 });
    expect(currentStep(steps, tied)).toBe(steps.length);
  });

  it('goes back to the step that needs a crossing that is undone',
    () => {
    const tied = recognise({ points: sample(trefoil, 120), radius:
    0.02 });
    expect(currentStep(steps, tied)).toBe(steps.length);
    const undone = { ...tied, crossings: 1 };
    expect(currentStep(steps, undone)).toBe(1);
  });
});
```

□

64 · used in 69e

Chapter 12

Assembly

This is the only file that will carry `@o` directives. The concept chapters never name an output path. They define chunks, and this file routes those chunks to the files the build expects, so the layout can change without touching a single narrative.

Stage one routes the scaffold, the rope model, the topology, the renderer, and the page. Each route below names one file and the chunk that fills it.

12.1 Tooling and page

⟨ `app/package.json` 66a ⟩ ≡

⟨ *package manifest* 17 ⟩

□

66a

⟨ `app/vite.config.ts` 66b ⟩ ≡

⟨ *Vite configuration* 18a ⟩

□

66b

⟨ `app/tsconfig.json` 66c ⟩ ≡

⟨ *TypeScript configuration* 18b ⟩

□

66c

⟨ `app/index.html` 66d ⟩ ≡

⟨ *HTML shell* 19a ⟩

□

66d

⟨ app/src/index.css 67a ⟩ ≡

⟨ *Global styles* 19b ⟩

□

67a

⟨ app/src/main.tsx 67b ⟩ ≡

⟨ *Entry point* 19c ⟩

□

67b

⟨ app/src/App.tsx 67c ⟩ ≡

⟨ *UI: application* 52 ⟩

□

67c

⟨ app/src/ui/RopeStage.tsx 67d ⟩ ≡

⟨ *UI: rope stage* 46 ⟩

□

67d

⟨ app/src/lessons/record.ts 67e ⟩ ≡

⟨ *Lessons: record and stub* 44 ⟩

□

67e

⟨ app/src/anchors/bar.ts 67f ⟩ ≡

⟨ *Anchors: bar* 45 ⟩

□

67f

12.2 Rope model

⟨ app/src/rope/snapshot.ts 67g ⟩ ≡

⟨ *Rope: snapshot types* 21 ⟩

□

67g

⟨ app/src/rope/rope.ts 68a ⟩ ≡

⟨ *Rope: physics model* 23 ⟩

□

68a

⟨ app/src/rope/rope.test.ts 68b ⟩ ≡

⟨ *Rope: tests* 26 ⟩

□

68b

12.3 Topology

⟨ app/src/topology/crossings.ts 68c ⟩ ≡

⟨ *Topology: crossings and Fox matrix* 32 ⟩

□

68c

⟨ app/src/topology/determinant.ts 68d ⟩ ≡

⟨ *Topology: determinant* 35 ⟩

□

68d

⟨ app/src/topology/recogniser.ts 68e ⟩ ≡

⟨ *Topology: recogniser* 36 ⟩

□

68e

⟨ app/src/topology/topology.test.ts 68f ⟩ ≡

⟨ *Topology: tests* 37 ⟩

□

68f

12.4 Recogniser, stage two

⟨ app/src/topology/alexander.ts 68g ⟩ ≡

⟨ *Recogniser: Alexander polynomial* 54 ⟩

□

68g

⟨ app/src/topology/library.ts 69a ⟩ ≡

⟨ *Recogniser: taught library* 57 ⟩

□

69a

⟨ app/src/anchors/shapes.ts 69b ⟩ ≡

⟨ *Recogniser: anchors as components* 58 ⟩

□

69b

⟨ app/src/topology/stage-two.test.ts 69c ⟩ ≡

⟨ *Recogniser: stage two tests* 59 ⟩

□

69c

12.5 Renderer

12.6 Guided mode

⟨ app/src/lessons/guided.ts 69d ⟩ ≡

⟨ *Guided: step check* 63a ⟩

□

69d

⟨ app/src/lessons/guided.test.ts 69e ⟩ ≡

⟨ *Guided: tests* 64 ⟩

□

69e

⟨ app/src/ui/GuidedPanel.tsx 69f ⟩ ≡

⟨ *UI: guided panel* 63b ⟩

□

69f

⟨ app/src/render/ropeTube.ts 70a ⟩ ≡

⟨ *Renderer: rope tube* 40 ⟩

□

70a

⟨ app/src/render/ropeTube.test.ts 70b ⟩ ≡

⟨ *Renderer: tube tests* 42 ⟩

□

70b